College of Computer Science and Information Technology
Department of Computer Engineering

ARTI 404 – Image Processing

Lab 4 Report: Intensity Transformations and Filtering – Spatial Domain

Student Name Mohammed Omar Halawani

Student ID 2240001174

Academic Year 2025 – 2026

1. Objective

This lab applies intensity transformations based on the image histogram: contrast stretching, histogram equalization and histogram matching. All of them were implemented in Python with scikit-image, NumPy and Matplotlib.

2. Images Used

All images come from `skimage.data`, as the lab specifies, and the notebook saves a copy of each to the `lab4/images/` folder:

- **moon.png** – grayscale lunar surface, used in Tasks 1 and 2.
- **rocket.png** – RGB image, used as the reference in Task 3.
- **chelsea.png** – RGB image of a cat, used as the source in Task 3.

3. Setup

```
import os
import numpy as np
import matplotlib.pyplot as plt
from skimage import data, exposure, io

img = data.moon()
reference = data.rocket()
source = data.chelsea()

os.makedirs("images", exist_ok=True)
io.imsave("images/moon.png", img)
io.imsave("images/rocket.png", reference)
io.imsave("images/chelsea.png", source)
```

4. Task 1 – Contrast Stretching (3rd–80th Percentile)

The 3rd and 80th percentiles of the moon image are computed with `np.percentile`. They are passed as `in_range` to `exposure.rescale_intensity`, which maps that intensity range linearly onto the full output range. Values below the 3rd percentile become black, and values above the 80th percentile become white.

```
p3, p80 = np.percentile(img, (3, 80))
img_rescale = exposure.rescale_intensity(img, in_range=(p3, p80))

fig = plt.figure(figsize=(10, 7))

fig.add_subplot(2, 1, 1)
plt.imshow(img_rescale, cmap="gray")
plt.axis("off")
plt.title("Contrast Stretch (3rd - 80th percentile)")

fig.add_subplot(2, 1, 2)
plt.hist(img_rescale.flat, bins=256, range=(0, 255))
plt.xlabel("gray level values")
plt.ylabel("Number of pixels")

plt.show()
```

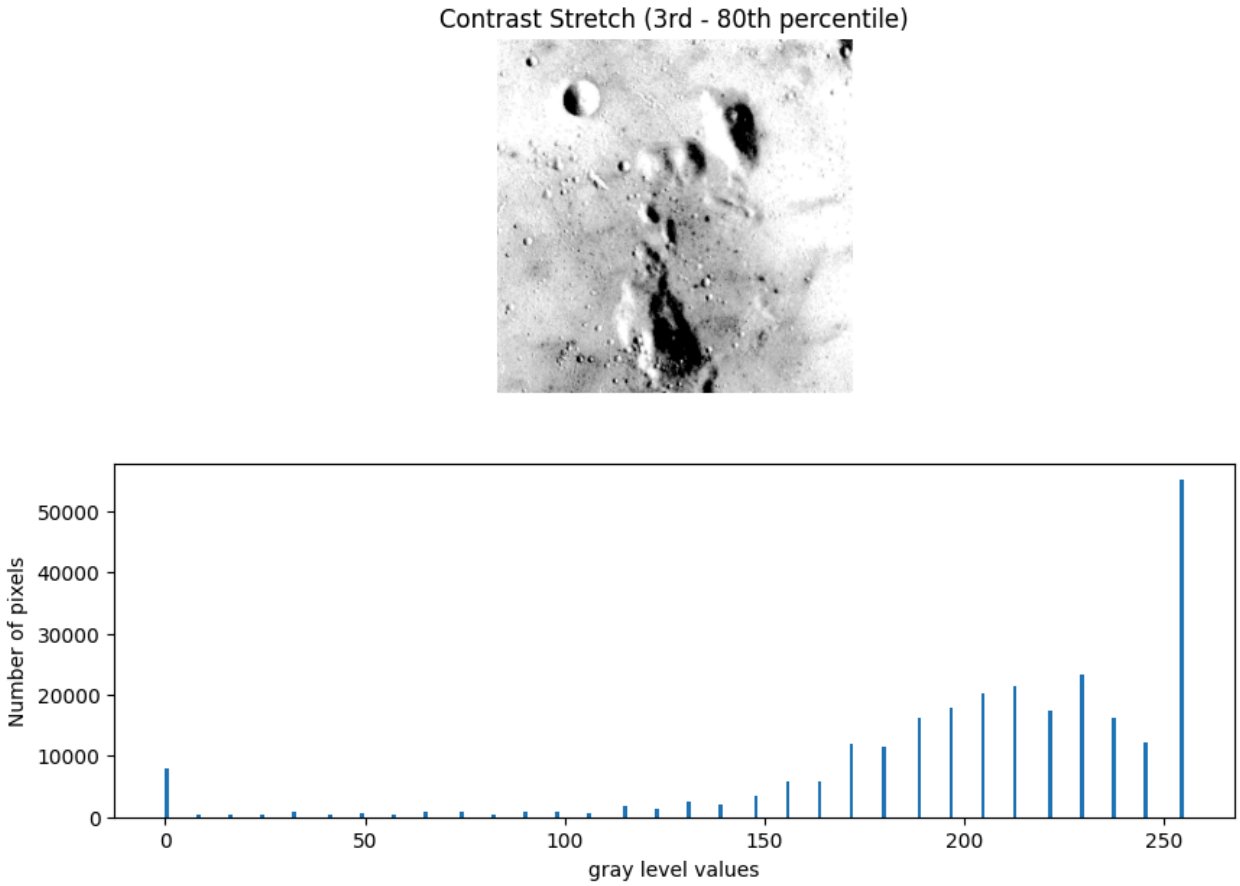


Figure 1: Moon image after contrast stretching (3rd–80th percentile) and its histogram.

Discussion: The original moon image covers only a narrow band of gray levels. After stretching, those levels spread across the whole 0–255 range, which is why the histogram looks like a comb of separate bars. Because the upper bound is only the 80th percentile, roughly 55,000 pixels are clipped to pure white (the tall bar at 255), and about 8,000 are clipped to black at 0. The image becomes much brighter and the dark craters stand out strongly, but detail in the bright regions is lost to saturation.

5. Task 2 – Histogram Equalization

`exposure.equalize_hist` uses the cumulative distribution function of the image as the mapping, so that the output intensities are spread as evenly as possible. The function returns floats in [0, 1], so the result is scaled back to 8-bit before the histogram is plotted on the 0–255 axis.

```
img_eq = exposure.equalize_hist(img)
img_eq = np.uint8(img_eq * 255)

fig = plt.figure(figsize=(10, 7))

fig.add_subplot(2, 1, 1)
plt.imshow(img_eq, cmap="gray")
plt.axis("off")
plt.title("Histogram Equalization")

fig.add_subplot(2, 1, 2)
plt.hist(img_eq.flat, bins=256, range=(0, 255))
plt.xlabel("gray level values")
plt.ylabel("Number of pixels")

plt.show()
```

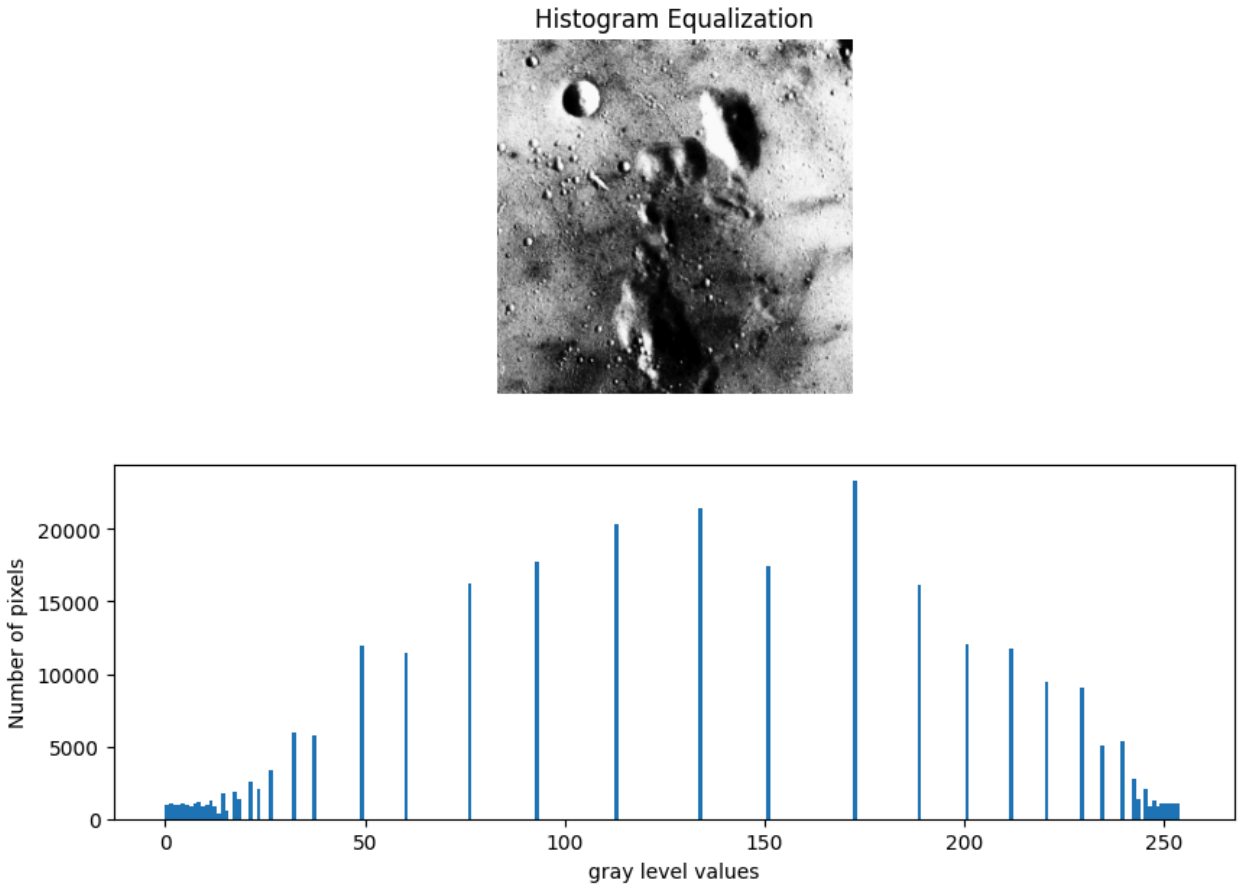


Figure 2: Moon image after histogram equalization and its histogram.

Discussion: Equalization uses the full gray-level range without clipping a large group of pixels, unlike Task 1. The result has balanced contrast: the craters, ridges and surface texture are all clearly visible in both dark and bright areas. The histogram is spread over 0–255 but is not perfectly flat. Equalization can only move whole gray levels and cannot split them, so the original levels become separate, widely spaced bars, while the sparse levels at each end are packed closely together.

6. Task 3 – Histogram Matching

`exposure.match_histograms` changes the source image (Chelsea) so that the histogram of each color channel matches the matching channel of the reference image (rocket). Setting `channel_axis=-1` makes the R, G and B channels match independently.

```
matched = exposure.match_histograms(source, reference, channel_axis=-1)
matched = np.uint8(np.clip(matched, 0, 255))

fig, axes = plt.subplots(1, 3, figsize=(15, 5))

axes[0].imshow(source)
axes[0].set_title("Source (Chelsea)")

axes[1].imshow(reference)
axes[1].set_title("Reference (Rocket)")

axes[2].imshow(matched)
axes[2].set_title("Matched")

for ax in axes:
    ax.axis("off")

plt.show()
```



Figure 3: Source, reference and histogram-matched images.

The per-channel histograms of the source, reference and matched images are then plotted to check the result:

```
colors = ("red", "green", "blue")
images = (source, reference, matched)
titles = ("Source", "Reference", "Matched")

fig, axes = plt.subplots(3, 3, figsize=(15, 9))

for row, channel_color in enumerate(colors):
    for col, (image, title) in enumerate(zip(images, titles)):
        axes[row, col].hist(image[..., row].flat, bins=256, range=(0, 255),
color=channel_color)
        axes[row, col].set_title(f"{title} - {channel_color} channel")

plt.tight_layout()
plt.show()
```

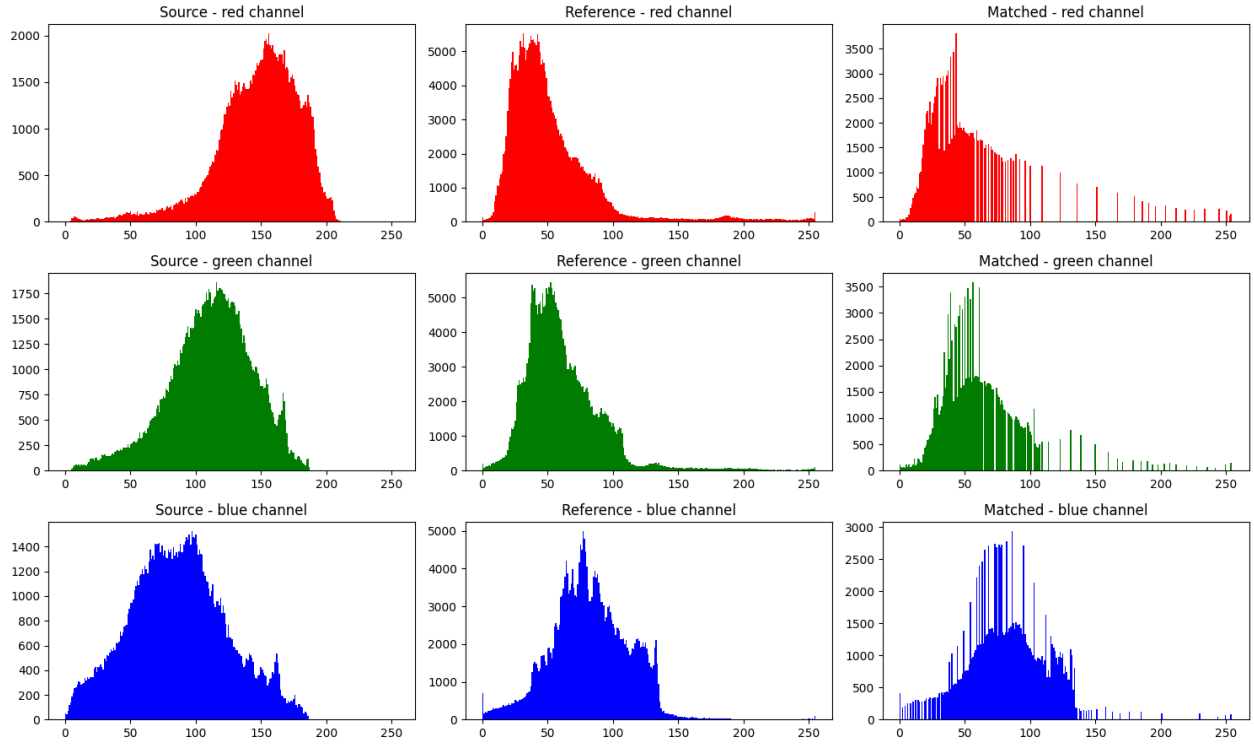


Figure 4: Red, green and blue channel histograms of the source, reference and matched images.

Discussion: The matched image keeps the structure of the Chelsea image but takes on the dark blue night-time tone of the rocket image. In the histograms, the source's bright red and green channels (peaking around 150 and 115) are pulled down to the low values of the reference (peaking around 40 and 55), and each matched channel follows the overall shape of the reference histogram. The matched histograms are spiky because the source's discrete gray levels are spread out to fit the reference distribution. Some red patches appear on the nose and fur. They come from the small tail of very bright red values in the reference (the launch lights) being assigned to the brightest red pixels of the source. Each channel is matched separately, so the colors of these pixels do not stay consistent with each other.

7. Conclusion

Contrast stretching, histogram equalization and histogram matching all remap pixel intensities using information from the histogram. Contrast stretching is simple and controllable, but with a narrow percentile range such as 3–80% it saturates a large part of the image. Equalization improves contrast automatically and gave the most balanced result on the moon image. Histogram matching transfers the tonal and color characteristics of one image to another, but matching each channel separately can produce color artifacts.

8. References

- Sandipan Dey, *Hands-On Image Processing with Python*, Packt.
- scikit-image: Histogram Equalization – scikit-image.org/docs/stable/auto_examples/color_exposure/plot_equalize.html
- scikit-image: Histogram Matching – scikit-image.org/docs/0.24.x/auto_examples/color_exposure/plot_histogram_matching.html